

АКОС

ИДЗ №1

Вариант 39. Регулируемый перекрёсток

Выполнил: студент БПИ252 Гришин Алексей Андреевич

Содержание

1	Реализация и интерфейс	2
1.1	Структура программы	2
1.2	Сборка и запуск	2
2	Проверки и реальные результаты	3
2.1	Ручной эталон <code>hand</code>	3
2.2	Автоматизированное тестирование	3
3	Требования, стиль и критическая оценка	4
3.1	Использование ИИ и ручные правки	4
A	Пользовательские промпты	5

1 Реализация и интерфейс

1.1 Структура программы

Архитектура модели разделена на несколько логических модулей для удобства поддержки и расширения:

Файл / Структура	Назначение
<code>main.c</code>	Точка входа. Обработка аргументов командной строки, перехват POSIX-сигналов (<code>SIGINT</code> , <code>SIGTERM</code>), инициализация и безопасное завершение.
<code>config.c</code> / <code>Config</code>	Парсер конфигурации. Инициализация потоков, направлений, фаз светофора, матрицы конфликтов и валидация входных данных.
<code>simulation.c</code>	Ядро симуляции. Содержит логику <code>Controller</code> , управление очередями (<code>Flow</code>), обработку прибытий и проездов, а также строгую проверку инвариантов.
<code>io.c</code> / <code>Output</code>	Обертки для POSIX-совместимого ввода-вывода (<code>open</code> , <code>read</code> , <code>write</code>) с обработкой системных прерываний (<code>EINTR</code>).
<code>crossroad.h</code>	Заголовочный файл с общими интерфейсами и структурами данных.

Основной цикл программы реализует потактовую событийную имитацию. На каждом такте последовательно выполняются: 1. Обработка завершённых проездов. 2. Проверка условий выхода (достижение лимитов или прерывание). 3. Генерация новых прибытий участников. 4. Обновление состояния светофорного контроллера. 5. Запуск новых машин на перекресток по правилам FIFO. 6. Аппаратная проверка инвариантов безопасности и вывод состояния в журнал.

Светофорный контроллер реализован как конечный автомат с состояниями:

$$\text{GREEN} \rightarrow \text{DRAIN} \rightarrow \text{CLEARANCE} \rightarrow \text{GREEN}$$

В состоянии `DRAIN` въезд на перекрёсток уже запрещён, но зелёный сигнал сохраняется для тех машин, которые завершают манёвр. Состояние `CLEARANCE` обеспечивает обязательный защитный интервал безопасности (всем горит красный).

В программе реализована как фиксированная, так и адаптивная стратегия управления. Адаптивный алгоритм в первую очередь обслуживает потоки с экстренным транспортом (без нарушения локального FIFO), а затем выбирает фазы на основе максимального времени ожидания и длины очередей.

1.2 Сборка и запуск

Проект собирается с помощью CMake. Команды для сборки:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
```

Запуск осуществляется командой: `./build/crossroad CONFIG [LOG]`. Вся конфигурация перекрёстка загружается из единого текстового файла (синтаксис описан в `README.md`). Использование файла конфигурации позволило гибко настраивать интенсивность потоков, матрицу конфликтов, зеленые фазы и общие параметры симуляции (тайминги, сид `rand`, стратегию).

Программа поддерживает два штатных сценария завершения: 1. По достижению лимита времени или целевого количества проехавших машин. 2. Принудительное завершение через системные сигналы `SIGINT` (Ctrl+C) или `SIGTERM`.

Во всех случаях программа корректно освобождает ресурсы, дописывает логи и выводит итоговую статистику по каждому направлению.

2 Проверки и реальные результаты

Тестирование проводилось в целевом окружении Linux (Debian 12, GCC), а также локально на macOS (Apple Silicon M4, Clang). Программа успешно проходит сборку с включенными санитайзерами памяти (ASan, UBSan) без предупреждений.

Ниже приведена сводная статистика по тестовым конфигурациям (Среднее и Максимум указывают время ожидания до въезда на перекресток):

Сценарий	Такт	Прибыло	Прошло	Отказы	Ждут	В зоне	Среднее	Макс.
demo	100	76	44	7	25	0	21,89	48
hand	8	16	3	10	2	1	2,00	5
emergency	20	40	7	1	32	0	7,71	15
compatible	6	12	4	0	6	2	0,50	1

2.1 Ручной эталон hand

Для верификации работы очередей и таймингов фаз был разработан детерминированный сценарий `hand.conf`. В нём каждый такт со 100% вероятностью генерируются автомобиль (поток А) и пешеход (поток Р).

Такт	Описание события
0	Автомобиль #1 и пешеход #2 встают в очереди; авто #1 начинает проезд.
2	Авто #1 покидает перекресток (ожидание 0); авто #3 начинает проезд.
4	Авто #3 покидает перекресток (ожидание 1); фаза авто завершена, запуск защитного интервала.
5	Начинается пешеходная фаза; пешеход #2 начинает переход (ожидание 5).
7	Пешеход #2 завершает переход; пешеход #14 начинает переход.
8	Завершение симуляции. Итог: 16 прибытий, 3 выхода, 10 отказов.

Результаты лог-файла полностью совпали с ручным расчётом, что аппаратно подтверждает правильность работы алгоритма FIFO и корректность обработки интервалов безопасности.

2.2 Автоматизированное тестирование

Дополнительно был разработан комплексный Python-скрипт `test_scenarios.py`, который парсит текстовые логи и независимо восстанавливает состояние перекрестка на каждом такте. Скрипт программно доказывает отсутствие коллизий конфликтующих направлений, честность очередей и корректность обработки системных прерываний. Тесты покрывают краевые ситуации, включая работу при переполненном диске (имитация через `/dev/full`) и обрыв канала `stdout`.

3 Требования, стиль и критическая оценка

Критерий	Реализация
Основные 1, 6	ФИО, группа, вариант, описание сценария и интерфейса представлены в отчете.
Основные 2-5	Модель перекрёстка реализована на C17. Используются системные вызовы POSIX, реализованы оба варианта завершения программы.
Доп. 1	Код снабжен подробными комментариями на русском языке, объясняющими архитектурные решения.
Доп. 2	Ввод-вывод полностью реализован через дескрипторы (<code>open</code> , <code>read</code> , <code>write</code>).
Доп. 3, 4	Программа сохраняет подробный лог и статистику в файл по окончании работы (в том числе при прерывании по <code>Ctrl+C</code>).
Доп. 5	Подготовлены тестовые сценарии (конфиги) и скрипт автоматизации <code>run.sh</code> .
Доп. 6, 7	Описание использования LLM и промпты приведены в отчете.

3.1 Использование ИИ и ручные правки

В ходе работы над проектом я использовал ИИ (LLM-ассистента) в качестве помощника для написания рутинного кода и первичного проектирования.

Критическая оценка и рефакторинг: Первичный код, предложенный нейросетью, содержал избыточные абстракции. Я провел глубокий рефакторинг, в результате которого оставил только необходимую и надежную логику, работающую через системные вызовы.

Также в процессе отладки я обнаружил и исправил критический баг (race condition) в сгенерированной логике перехвата **SIGTERM**, из-за которого происходила потеря финальной статистики. Для гарантии надежности итогового решения я самостоятельно спроектировал тестовый контур на Python, который строго верифицирует текстовый выхлоп C-программы.

В качестве ограничений текущей имитационной модели стоит отметить её дискретность: за один такт генерируется не более одного участника на поток, отсутствуют перестроения, обгоны внутри полосы и не моделируется поведение на желтый сигнал светофора.

А Пользовательские промпты

Ниже приведены запросы, использованные для коммуникации с ИИ-ассистентом на этапе генерации каркаса программы.

Промпт 1

Я пишу потактовую симуляцию на C17 под Linux. У меня есть основной цикл `while(1)`. Как правильно перехватить сигналы `SIGINT` и `SIGTERM` через `sigaction`, чтобы безопасно выставить атомарный флаг и выйти из цикла? Также напиши обертку для системного `write`, которая гарантированно дописывает буфер и корректно обрабатывает прерывание `errno == EINTR`.

Промпт 2

В моей модели светофора есть до 16 потоков. Я храню текущую разрешающую фазу как битовую маску `uint32_t mask`. У каждого потока `i` есть своя маска несовместимых направлений `conflicts[i]`. Напиши лаконичную и быструю функцию на побитовых операциях, которая проверяет, что в текущей фазе `mask` нет ни одной пары конфликтующих потоков.

Промпт 3

Мне нужно распарсить текстовый файл конфигурации (читаю через `open/read` в буфер). Формат строк: `stream N car north straight 8 2 180`. Сторонние библиотеки нельзя. Напиши надежный парсер строк с использованием `strtok_r` и функции безопасного чтения чисел `strtoull`. Парсер должен строго реагировать на переполнения и мусорные символы.

Промпт 4

Помоги спроектировать конечный автомат для контроллера перекрестка. Состояния: `GREEN`, `DRAIN` (въезд запрещен, но зеленый горит, пока перекресток не покинут уже въехавшие машины), `CLEARANCE` (защитный красный для всех). Опиши логику переходов между этими состояниями на каждом такте времени, учитывая параметры `min_green`, `max_green` и `clearance`. Напиши только скелет логики (псевдокод или C).

Промпт 5

Моя Си-программа выводит лог событий: `[t=14] Въезд: поток=W id=1 длина=1`. Напиши Python-скрипт через модуль `subprocess`, который запускает бинарник и парсит его `stdout` регулярками на лету. Скрипт должен проверять, что `id` машин в рамках одного потока всегда идут строго по возрастанию (FIFO), и что одновременно на перекрестке не находятся машины из конфликтующих потоков.

Промпт 6

Напиши `CMakeLists.txt` для проекта на C. Нужно включить стандарт C17, определить макрос `_POSIX_C_SOURCE=200809L` и добавить строгие флаги компилятора: `-Wall -Wextra -Wpedantic -Wconversion -Wshadow`. Также добавь опцию сборки с санитайзерами (ASan, UBSan) для компиляторов GCC и Clang.

Промпт 7

Сделай изображение регулируемого перекрестка в стиле vintage comic style.